DSA I (CSE 2215) – Recursion

Introduction to Recursion Loop Complexities

Asif Abrar, Lecturer of CSE

UIU

What is Recursion?

Recursion means a function calling itself to solve a problem.

- Problem is divided into smaller versions of itself.
- Each call solves a smaller part.
- Stops when a **base case** is reached.
- Useful in problems with natural substructure.

$$\textbf{Recursive function} = \textbf{Base Case} + \textbf{Recursive Case}$$

Simple Example: Print Numbers from 1 to n

Code:

```
void print1toN(int n){
    if(n == 0) return;        // Base case
    print1toN(n-1);           // Recursive call
    cout << n << " ";
}
```

Intuition: Solve for 1..(n-1) first, then print n.

Sum of First n Numbers

Code:

```
int sum(int n){
    if(n == 0) return 0;    // Base case
    return n + sum(n-1);    // Recursive call
}
```

Intuition:

$$sum(n) = n + sum(n-1)$$

Each call does a small part and waits for the rest to finish.

Factorial Example

Definition: $n! = n \times (n-1)!$ and $0! = 1$

Code:

```
int factorial(int n){
    if(n == 0) return 1;  // Base case
    return n * factorial(n-1);
}
```

Intuition: Let recursion handle repeated multiplication.

Fibonacci Example

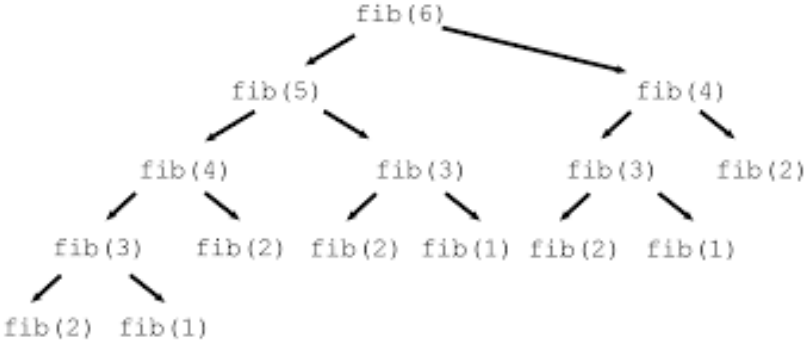
Definition: $F(n) = F(n-1) + F(n-2)$, $F(0) = 0, F(1) = 1$

Code:

```
int fib(int n){
    if(n <= 1) return n;            // Base case
    return fib(n-1) + fib(n-2);     // Recursive case
}
```

Intuition: Each call splits into two smaller calls $\rightarrow$ recursion tree.

Recursion Tree Visualization



Stack Simulation – sum(3)

Function: $sum(n) = n + sum(n-1)$

Call: sum(3)

$$\begin{aligned}
&\text{sum}(3) \text{ calls sum}(2)\\
&\quad \text{sum}(2) \text{ calls sum}(1)\\
&\qquad \text{sum}(1) \text{ calls sum}(0)\\
&\qquad\quad \text{sum}(0) = 0 \text{ (base case)}
\end{aligned}$$

Return Phase:

$$\begin{aligned}
\text{sum}(1) &= 1 + 0 = 1\\
\text{sum}(2) &= 2 + 1 = 3\\
\text{sum}(3) &= 3 + 3 = 6
\end{aligned}$$

Iteration – Sum of First n Numbers

Iterative Code:

```
int sum_iter(int n){
    int s = 0;
    for(int i=1; i<=n; i++)
        s += i;
    return s;
}
```

Points:

- Uses loop
- Executes in one continuous flow
- Memory: $O(1)$

Recursion – Sum of First n Numbers

Recursive Code:

```
int sum_rec(int n){
    if(n == 0) return 0;          // Base case
    return n + sum_rec(n-1);      // Recursive case
}
```

Points:

- Each call stored in call stack
- Ends at base case
- Memory: $O(n)$
- Expressive for divide-and-conquer problems

Recursion Example – Factorial

Recursive Code:

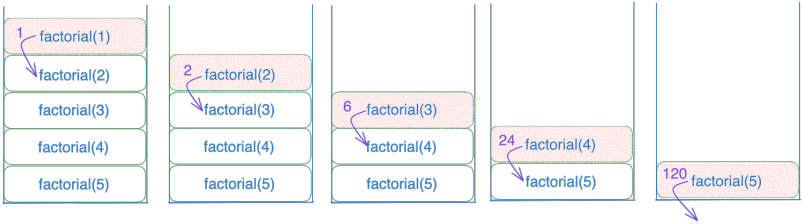
```
int fact(int n){
    if(n == 0) return 1;    // base case
    return n * fact(n-1);   // recursive call
}
```

Call Trace:

$$\text{fact}(4) \to 4 \cdot 3 \cdot 2 \cdot 1$$

Observation: Each call waits for the next $\to$ stack grows.

Recursion Call Stack Visualization



What is Iteration?

Definition: Using loops (`for`, `while`) to repeat an operation until a condition is met.

$$\text{Iteration} = \text{repeat} \rightarrow \text{update} \rightarrow \text{stop}$$

Characteristics:

- Uses loop control variables
- Does not use function call stack
- Generally more memory-efficient

Iteration Example – Factorial

Iterative Code:

```
int fact(int n){
    int result = 1;
    for(int i = 1; i <= n; i++){
        result *= i;
    }
    return result;
}
```

Observation: Updates value inside loop → no function calls.

Recursion vs Iteration – Comparison

Feature	Recursion	Iteration
Memory Usage	High (stack frames)	Low
Time Complexity	Often equal	Often equal
Termination	Base case	Loop condition
Overhead	Function calls	No overhead
Ease of Use	Good for tree/graph	Good for loops
Risk	Stack overflow	Infinite loop

When to Use Which?

Use Recursion When:

- Natural recursive structure (trees, DFS)
- Divide-and-conquer (merge sort, quick sort)
- Code simplicity is more important

Use Iteration When:

- Memory constraints are important
- Simple repetitive operations
- Stack depth may be large

Summary

- Recursion uses self-calls and stack $\rightarrow$ elegant but memory-heavy
- Iteration uses loops $\rightarrow$ efficient but sometimes less clear
- Both can solve similar problems but choice depends on:
  - efficiency
  - clarity
  - memory usage
  - problem structure

Single Loop Complexity

Example 1: Simple loop

```
for(int i = 0; i < n; i++){
    // constant time operation
}
```

Analysis: - Loop runs exactly n times $\rightarrow O(n)$ - Each operation inside is $O(1)$

Loop with Multiplicative Increment

Example 2: i multiplied each time

```
for(int i = 1; i < n; i *= 2){
    // constant time operation
}
```

Analysis: - i = 1, 2, 4, 8, . . . $<$ n $\rightarrow$ $\log_2 n$ iterations - Complexity $\rightarrow$ $O(\log n)$

Nested Loops – Quadratic Example

Example 3: Nested loops

```
for(int i = 0; i < n; i++){
    for(int j = 0; j < n; j++){
        // constant operation
    }
}
```

Analysis: - Outer loop runs n times, inner loop runs n times each → n*n iterations - Complexity → $O(n^2)$

Nested Loops with Quadratic Increment

Example 4: j squared increment

```
for(int i = 0; i < n; i++){
    for(int j = 1; j*j < n; j++){
        // operation
    }
}
```

Analysis: - Inner loop runs until j*j < n → j < sqrt(n) - Total iterations → n * sqrt(n) → $O(n\sqrt{n})$

Loops with Different Increments

Example 5: Linear multiplication increment

```
for(int i = 0; i < n; i += 3){
    // operation
}
```

Analysis: - i = 0, 3, 6, ... < n $\rightarrow$ n/3 iterations - Complexity $\rightarrow O(n)$ (constant factor ignored in Big-O)

Nested Loops – Cubic Example

Example 6: Triple nested loop

```
for(int i = 0; i < n; i++){
    for(int j = 0; j < n; j++){
        for(int k = 0; k < n; k++){
            // operation
        }
    }
}
```

Analysis: - Each loop runs n times $\rightarrow$ n * n * n = $n^3 iterations - Complexity ß O(n^3)$

Intuition Summary

- Single loop $\rightarrow O(n)$
- Multiplicative increment (i*=2) $\rightarrow O(\log n)$
- Nested loops $\rightarrow$ multiply sizes: n*n $\rightarrow O(n^2)$, n*n*n $\rightarrow O(n^3)$
- Loops with j*j or sqrt limit $\rightarrow$ sub-linear inner $\rightarrow$ n*sqrt(n)
- Constant factor increments (i+=3) $\rightarrow$ same Big-O as n

Array Mapping – Introduction

Goal: Convert logical array index (i, j) into a physical memory address.

$$\text{Address} = b + (\text{offset})$$

Where:

- b = base address of the array in memory
- L = size (in bytes) of each element
- l_1, u_1 = lower and upper bounds of row index
- l_2, u_2 = lower and upper bounds of column index

Mapping 1D Array

General Formula:

$$\text{addr}(A[i]) = b + (i - l_1) \cdot L$$

Meaning:

- Skip $(i - l_1)$ elements
- Each element takes L bytes

Example Memory Layout (1D):

Index	Memory Cell
l_1	b
$l_1 + 1$	$b + L$
$l_1 + 2$	$b + 2L$
$\vdots$	$\vdots$
u_1	$b + (u_1 - l_1)L$

Mapping 2D Array – Row Major Order (C/C++/Java)

General Formula:

$$\text{addr}(A[i,j]) = b + (i - l_1)(u_2 - l_2 + 1)L + (j - l_2)L$$

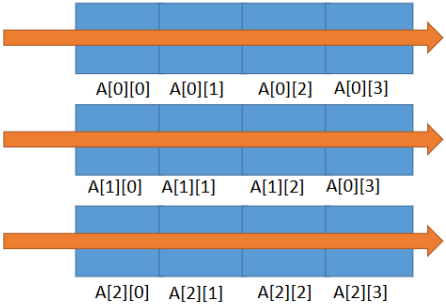
Explanation:

- First skip $(i - l_1)$ full rows
- Each row contains $(u_2 - l_2 + 1)$ columns
- Then skip $(j - l_2)$ columns inside the row

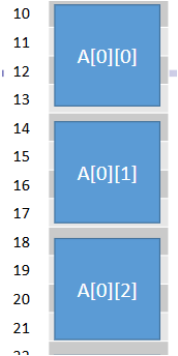
Row-Major Memory Layout:

Element	Stored In Memory As
$A[l_1, l_2]$	b
$A[l_1, l_2 + 1]$	$b + L$
$A[l_1, l_2 + 2]$	$b + 2L$
$\vdots$	$\vdots$
$A[l_1 + 1, l_2]$	$b + (u_2 - l_2 + 1)L$

Row-Major Visualization



Row-Major Layout



Row-Major in Memory

Mapping 2D Array – Column Major Order (Fortran, MATLAB)

General Formula:

$$\text{addr}(A[i,j]) = b + (j - l_2)(u_1 - l_1 + 1)L + (i - l_1)L$$

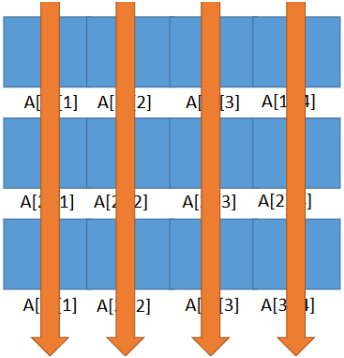
Explanation:

- First skip $(j - l_2)$ full columns
- Each column contains $(u_1 - l_1 + 1)$ rows
- Then skip $(i - l_1)$ rows inside the column

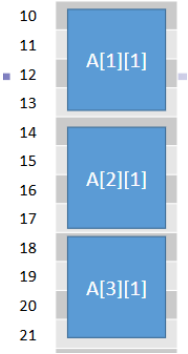
Column-Major Memory Layout:

Element	Stored In Memory As
$\text{A}[l_1, l_2]$	b
$\text{A}[l_1 + 1, l_2]$	$b + L$
$\text{A}[l_1 + 2, l_2]$	$b + 2L$
$\vdots$	$\vdots$
$\text{A}[l_1, l_2 + 1]$	$b + (u_1 - l_1 + 1)L$

Column-Major Visualization



Column-Major Layout



Column-Major in Memory

Summary of Array Mapping

- 1D Array: $b + (i - l_1)L$
- 2D Row Major:
$$b + (i - l_1)(u_2 - l_2 + 1)L + (j - l_2)L$$
- 2D Column Major:
$$b + (j - l_2)(u_1 - l_1 + 1)L + (i - l_1)L$$
- $(u - l + 1)$ represents number of elements in an inclusive range.

Thank You!

Questions?